

WattGraphNet: Efficient Attention Based Spatial-Temporal Graph Convolutional Networks with Adaptive Adjacency Matrix Architecture for Virtual Power Plant Load Forecasting

Arsanchai Sukkuea , Pattaradanai Akkharat

School of Engineering and Technology, Walailak University, 222 Thaiburi, Thasala District,
Nakhon Si Thammarat 80160, Thailand.

Department of Computer Engineering, Faculty of Engineering, Chulalongkorn University 17th
floor, Engineering 4 Building (Charoenvidsavakham), Phayathai Road, Wang Mai, Pathumwan,
Bangkok, Thailand, Bangkok

Abstract

Virtual Power Plants (VPPs) aggregate distributed energy resources to operate as a single power entity, but accurate load forecasting for such systems remains challenging due to complex spatial-temporal dependencies. We present WattGraphNet, an attention-based spatial-temporal graph convolutional network (ASTGCN) architecture enhanced with an adaptive adjacency matrix and exported via ONNX for efficient inference. Using a real smart-campus microgrid dataset (Nov 2023–Nov 2024) from Chulalongkorn University in Thailand, our model achieves superior accuracy (SB-WAPE $\approx 19.11\%$, MAE ≈ 19.75 kW) compared to multiple benchmarks. The trained WattGraphNet model runs in under 0.1 seconds per sample on CPU (Middle Grade CPU), demonstrating its practicality for real-time VPP operation. These results highlight the potential of deep graph learning and model optimization for improving VPP load forecasting and deployment on edge devices.

Keywords: Virtual Power Plant (VPP), Based Spatial-Temporal Graph Convolutional Networks, Based Spatial-Temporal Graph Convolutional Networks, Based Spatial-Temporal Graph Convolutional Networks

1.Introduction

Global electricity demand has been steadily rising due to population growth, rapid urbanization, and the increasing adoption of energy-intensive technologies such as electric vehicles, air conditioning, and digital devices. This surge in consumption places significant pressure on existing power grids, making efficient energy management more critical than ever. Traditional centralized power generation is struggling to meet fluctuating demand while maintaining stability, especially as renewable energy sources with variable output—such as solar and wind—become a larger share of the energy mix. As a result, modern power systems are shifting toward smarter, more flexible architectures that can balance supply and demand in real time while integrating diverse energy resources.

Virtual Power Plants (VPPs) have emerged as a key concept in modern smart grids, integrating diverse distributed energy resources (DERs) – such as solar panels, battery storage, and controllable loads – into a coordinated network that can be managed as a single entity [3]. By pooling resources and flexibly controlling loads, VPPs enhance grid stability and allow participation in energy markets. Achieving these benefits, however, critically depends on precise load forecasting for the aggregated resources [3]. Accurate short-term load predictions enable the VPP to optimally dispatch generation, schedule storage usage, and trade energy, while poor forecasts can lead to suboptimal decisions or grid imbalances. Yet, load forecasting in a VPP context is challenging because it must capture not only temporal consumption patterns but also interactions between multiple distributed sites.

Traditional load forecasting methods for individual buildings or grids – ranging from statistical models to machine learning regressors – often struggle to model the complex nonlinear relationships and simultaneous dynamics present in VPPs. Each DER or load site in a VPP may have unique usage patterns and is influenced by common factors like weather and operational schedules. The spatial correlations between sites (for example, buildings on the same campus or feeders on the same distribution network) can be significant but are not explicitly accounted for in classical forecasting models. This motivates the application of deep learning approaches that can automatically learn both temporal patterns and spatial dependencies from data. Recurrent neural networks (RNNs) and transformers have been applied to time-series forecasting, but they usually treat multi-site data as a flat set of features, lacking an explicit representation of the network structure connecting VPP components. In contrast, graph-based deep learning methods provide a natural way to incorporate the connectivity or relationship between nodes in a power network.

Graph neural networks (GNNs) have shown success in domains like traffic forecasting by modeling sensor networks as graphs, where nodes are locations and edges represent road connections. In particular, the Attention-Based Spatial-Temporal Graph Convolutional Network (ASTGCN) introduced by Guo *et al.* [1] demonstrated that combining graph convolution layers (to capture spatial relationships) with temporal convolutions and attention mechanisms yields state-of-the-art accuracy for traffic flow prediction. This approach is promising for VPP load forecasting, since a VPP can be represented as a graph of loads (nodes) with certain connectivity (physical or inferred) between them. By leveraging attention mechanisms, the model can also focus on the most relevant time steps and most influential nodes when making a prediction. However, the original ASTGCN was designed for traffic networks and assumes a fixed known adjacency matrix of the graph. In a VPP, the strength of relationships between different sites may not be known a priori or static – e.g., two buildings might exhibit similar load patterns on weekdays (implying a strong connection) but diverge on weekends. This observation leads to our motivation for an adaptive adjacency matrix that the model can learn from the data, rather than relying on a manually defined graph structure.

In this work, we develop **WattGraphNet**, an improved ASTGCN-based architecture tailored for VPP load forecasting. Our contributions are threefold. First, we enhance the ASTGCN model by introducing a learnable adaptive adjacency matrix, allowing the network to automatically infer inter-node relationships and thus better capture the spatial correlations in the VPP’s loads. Second, we implement an efficient workflow for training and deploying the model, including data downsampling techniques to reduce training cost and exporting the trained model to the Open Neural Network Exchange (ONNX) format for optimized runtime inference [2]. Third, we design a comprehensive evaluation framework comparing our approach against a suite of baseline models (both traditional and deep learning) on a real-world smart campus dataset. In the following sections, we describe the dataset and methodology, present the results and efficiency measurements, and discuss the implications for VPP operations and future work.

2. Methods

2.1 Dataset

We evaluate our model using a smart-campus microgrid dataset from Chulalongkorn University (Bangkok, Thailand). The dataset comprises 12 months (November 2023 to November 2024) of electric load measurements recorded at 15-minute intervals from six nodes in the campus power network. These nodes include five academic or office buildings and one electric vehicle (EV) charging station, representing the diverse load types within the campus VPP. For convenience, we refer to the stations by their nicknames: *Chamchuri 4*, *Chamchuri 9* (building complexes), *Boromrajakumari* (a research building), *Julajakrapong* (an administrative building), and an *EV charging station*. Each node’s time series records its electrical demand in kilowatts (kW). Over the year-long span, the dataset captures typical university operational patterns, including weekday vs. weekend differences and semester breaks, as well as seasonal effects on building loads (e.g., cooling demand). The peak loads of the buildings vary widely (on the order of 160–660 kW), underscoring the heterogeneity that a VPP controller must handle. This multi-site dataset is well-suited to a graph-based modeling approach, since we expect certain groups of buildings to have correlated demand profiles (for instance, buildings with similar usage or those affected by common schedules). We partition the data into a training set and a testing set based on time: the model is trained on the first portion of the timeline (e.g., months 1–11) and tested on the last portion (e.g., the final month), to evaluate true forward-facing forecasting performance.

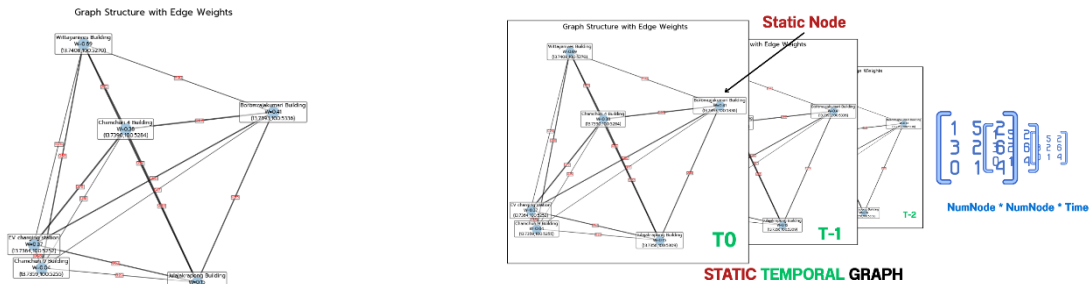
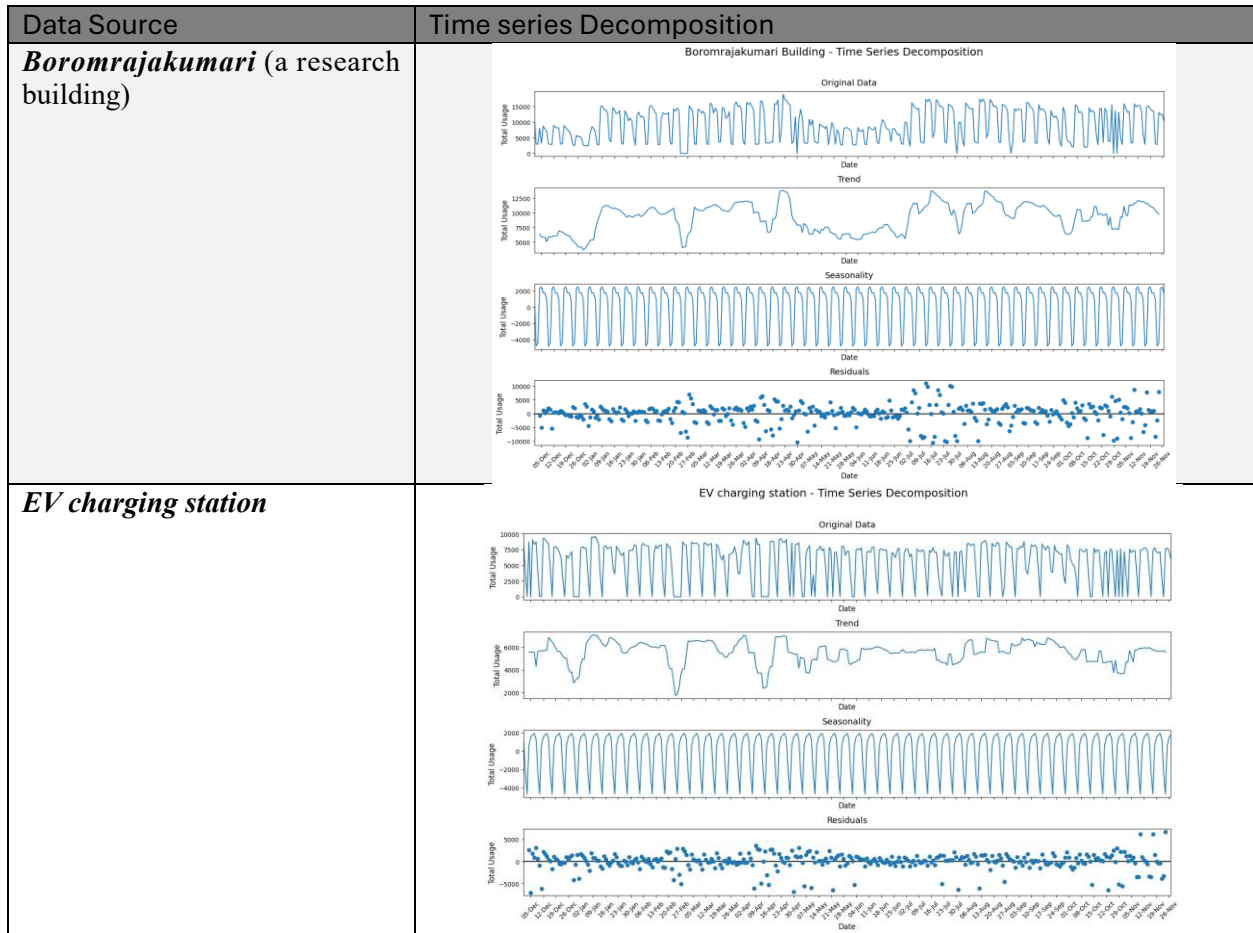


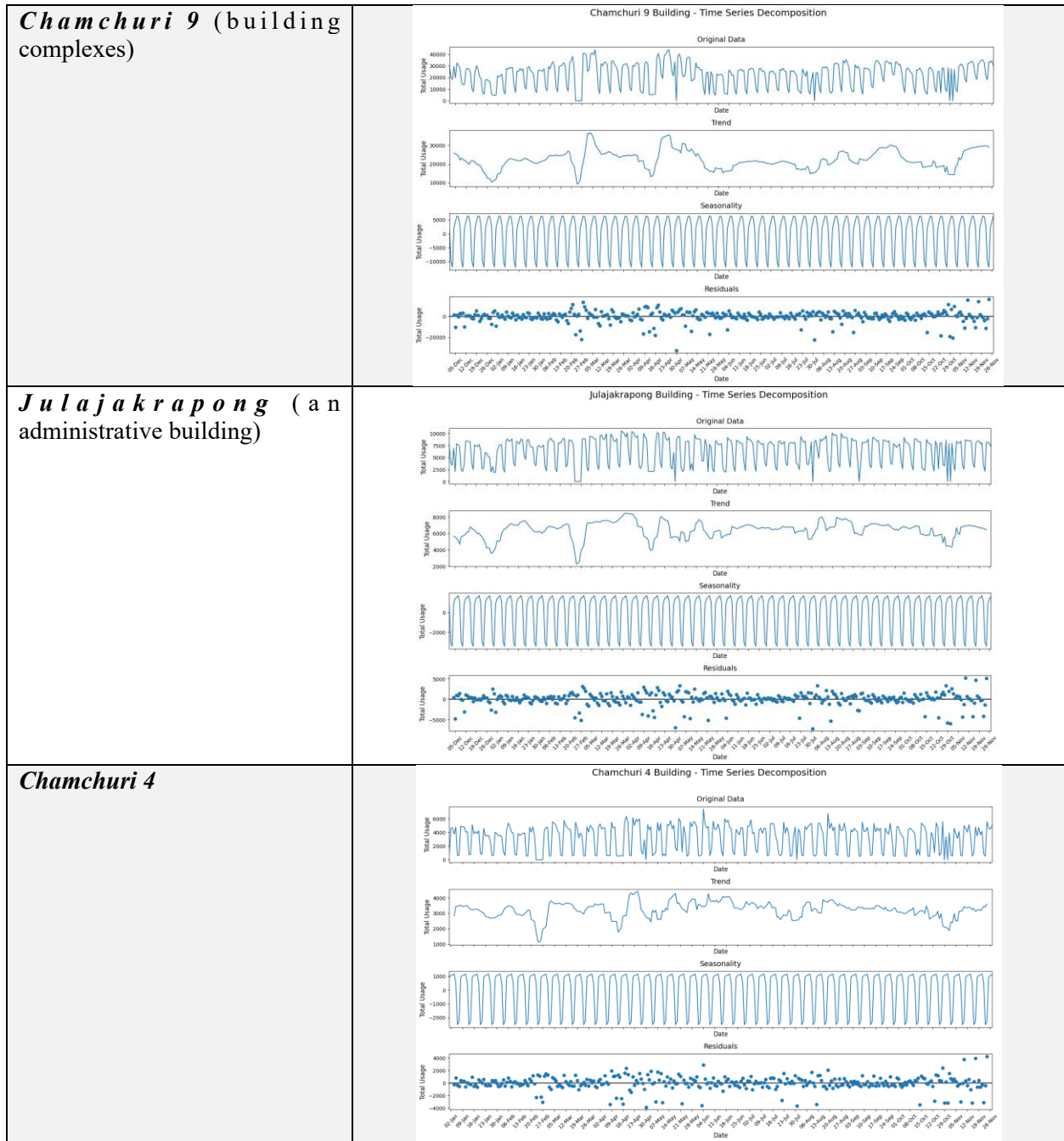
Figure 1 Our Graph data representation in temporal

Figure 2 Our Graph data representation in $t=0$

2.2 Data Preprocessing

Data cleaning and preprocessing were performed to ensure quality inputs for modeling. First, any obviously erroneous or negative consumption values were set to zero, as negative kW readings are not physically meaningful and typically indicate sensor or data entry errors. We conducted exploratory data analysis including seasonal-trend decomposition of each time series (expressing each load Y_t as $Y_t = T + S + R_t$, where T is a long-term trend, S is seasonal variation, and R_t is residual noise).





This analysis confirmed strong daily and weekly seasonality in the building loads, as expected in a campus environment (weekday daytime peaks vs. lower weekend usage), and a relatively stable trend over the one-year period.

To reduce the computational load for model training, we applied a down sampling strategy on the training data. In particular, we down sampled the training time series by 50%, using a 30-minute interval (every second data point) instead of the original 15-minute interval. This effectively halves the number of training samples and thus speeds up training and lowers memory usage. We chose a 50% down sampling because preliminary tests indicated that it yields only a minor accuracy degradation. In fact, we found that reducing the training data by half increased SB-WAPE by less than 3

percentage points in our case, a tolerable trade-off for the gains in efficiency. To maintain alignment with the 15-minute resolution needed for evaluation, the down sampled training sequences were up sampled back (with missing timestamps appropriately filled with forward fill) so that the model's input and output remain on the 15-minute grid. This approach leverages the insight that in a campus scenario, demand changes within a 15-minute window are not drastic (especially during mid-day steady states), so training on a slightly coarser time scale retains the important patterns while cutting the data size in half

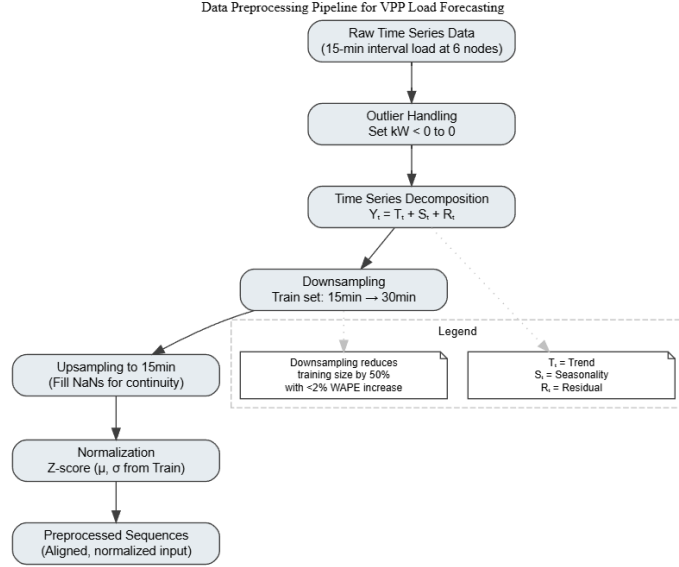


Figure 3: Data Preprocessing for VPP Load Forecasting

2.3 Evaluation

For performance evaluation, we split the timeseries data to train for 80% (from Start) and Test for 20% (to End), we use two common error metrics: Mean Absolute Error (MAE) and **Station-Balanced Weighted Absolute Percentage Error (SB-WAPE)**. MAE is defined as the average of the absolute errors,

$$MAE = \left(\frac{1}{N}\right) \sum_t |y_t - \hat{y}_t| ; \text{and has the units of the target (kW in this case)}$$

$$w_t = \frac{\max_t(N_t)}{N_j} ; \text{inverse-frequency weights (in each station)} ; \text{ where } N_j \text{ is the number of records for station } j,$$

$$SB\text{-}WAPE = \frac{(\sum_t w_t |y_t - \hat{y}_t|)}{(\sum_t w_t y_t)} \times 100\% ; \text{ The Station-Balanced Weighted Absolute Percentage Error (SB-WAPE) is a variant of the traditional WAPE metric designed to account for imbalanced sample sizes across stations.}$$

SB-WAPE is useful for aggregate demand forecasting since it normalizes the error with respect to the total load, providing a meaningful percentage of error relative to total demand. We report SB-WAPE for the overall forecast horizon and also examine MAE to understand the absolute error magnitude in kW. In our experiments, the forecasting task is to predict the next 24 hours of load for all six nodes (96 future 15-min steps for each node) given the past 24 hours of data. This rolling day-ahead forecasting setup reflects a practical scenario for VPP operations, where each day’s consumption profile is predicted from the recent history. The models are trained on historical data (as described in Section 2.1) and then generate multi-step forecasts on the held-out test period. We compare our proposed model’s forecasts against those from several baseline models (detailed in Section 2.4) using the same test set. All models receive the same input data segments and predict the same horizon, to ensure a fair comparison of forecasting accuracy. For statistical robustness, we evaluate errors over all days in the test set and report the aggregate SB-WAPE and MAE. We also track each model’s inference time and memory usage when generating forecasts on a standard CPU environment, as deployment efficiency is a key concern for VPP applications.

2.4 Traditional Model

We implemented a range of baseline forecasting methods, including both classical approaches and modern deep learning models, to serve as benchmarks against WattGraphNet. (i) Random Forest: As a representative machine learning baseline, we trained a Random Forest regressor (an ensemble of decision trees) to predict the next time step’s load for each node. Multi-step forecasts were produced recursively (feeding predicted values back in for subsequent steps). Random Forests can capture some nonlinearity but lack explicit temporal and spatial modeling, treating lagged loads as independent features. (ii) Seasonal Naïve: This is a simple but often effective time series baseline where the prediction is set equal to the value from the previous equivalent season. In our day-ahead context, the Seasonal Naïve forecast for each 15-min interval is simply the load from exactly one week prior (assuming weekly periodicity dominates) or one day prior if daily repetition is expected. We tested both variants and found weekly seasonality slightly more robust due to weekday/weekend patterns. (iii) Theta Method: We included a statistical forecasting method known as the Theta model (with optimized Theta coefficient), which is a decomposition approach that has performed well in M3 competition. We used a dynamic optimization of the Theta parameter to fit the training series; this method can be considered a form of exponential smoothing and linear extrapolation combination. (iv) Chronos (Automated DL): We evaluated an automated time-series forecasting toolkit (Chronos), which provides pre-trained deep learning models. Specifically, we used two modes from Chronos: a *fine-tuned* model where a pre-trained generic time-series model (nicknamed “bolt_small”) is further trained on our data, and a *zero-shot* model where a larger pre-trained model (“bolt_base”) is applied to our data without additional training. These represent advanced transfer-learning baselines that can leverage external data or architectures optimized by others. (v) PatchTST: We compared against PatchTST [4], a recent state-of-the-art transformer-based model for long-horizon time series forecasting.

PatchTST uses a channel-independent Transformer architecture with patching of time series [4], and has been shown to outperform standard Transformer models on various forecasting tasks. We trained PatchTST on our dataset (multi-variate input of all nodes) for the same 96-step horizon. This model explicitly does not incorporate graph structure, but its self-attention mechanism can implicitly learn some inter-series correlations.

2.5 Our Model Architecture

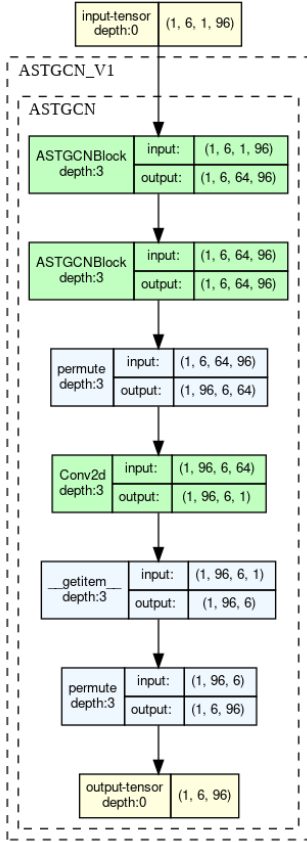


Figure 4 ASTGCN Core Block Architecture

Our proposed model, WattGraphNet, is built upon the ASTGCN framework [1] with significant enhancements to tailor it for VPP load forecasting. At its core, WattGraphNet is an attention-based spatial-temporal graph convolutional network that processes multivariate time series from multiple nodes (the VPP’s sites) simultaneously. Each node (building or station) is represented as a vertex in a graph, and the graph structure is learned by the model. The network comprises multiple stacked spatial-temporal blocks: each block contains a spatial graph convolution layer and a temporal convolution (or sequence modeling) layer, preceded by an attention mechanism. The spatial convolution uses graph convolutional filters to aggregate information from each node’s neighbors in the graph, effectively learning how the load at one site is influenced by loads at other sites. The temporal convolution (or in some implementations, an LSTM or attention across time) captures the sequence dynamics for each node. ASTGCN, as introduced in [1], also employs separate components to handle recent, daily-periodic, and weekly-periodic inputs – capturing different frequency patterns – and then fuses their outputs. In our implementation, we focus on the recent history component for simplicity and find that it suffices when the model has enough capacity, since weekly patterns are indirectly learned through the graph and attention over daily cycles.

A key innovation in WattGraphNet (Our) is the introduction of an Adaptive Adjacency Matrix (AAM) that is learned during training.

$$A_{adaptive} = \text{Softmax}(\text{ReLU}(E_1 E_2^T)) ; \text{ Where } E_1, E_2 \text{ are node embedding matrices.}$$

Instead of assuming a fixed adjacency based on physical connections or distance, we give the model the freedom to infer the strength of connection between any two nodes. We implement this by assigning each node two learnable embedding vectors. During a forward pass, we generate an $N \times N$ weight matrix $A_{adaptive}$ by multiplying the first embedding of each node with the second embedding of every other node (this can be seen as learning a low-rank representation of the

adjacency). We apply a ReLU activation to ensure non-negativity and then a row-wise softmax, yielding a normalized adjacency matrix where A_{ij} indicates the learned influence of node j on node i . This adaptive graph matrix is used by the graph convolution layer to filter and mix signals from different nodes.

AAM with GNN Spatial-Temporal Convolution Formula:

$$X^{(l+1)} = \sigma(\hat{A}_{adaptive} X^l W^l)$$

Where:

- $X^{(l)}$: input feature matrix at layer l .
- W^l : trainable weight matrix.
- $\hat{A}_{adaptive}$: Adaptive adjacency matrix.
- σ : activation function (e.g., ReLU).

The learning process adjusts these weights to minimize forecasting error, thereby discovering underlying relationships – for example, the model may learn high weights between two buildings that have synchronous usage patterns (like lecture halls that follow the same class schedule). This approach is akin to recent strategies in traffic forecasting and other GNN applications where the graph structure is not fixed but learned as part of the model training, improving flexibility and often accuracy. By incorporating AAM, WattGraphNet can automatically identify and emphasize correlations such as “*the EV charging station load often spikes at the same time as Building X’s load*”, even if such relations were not pre-defined by the user.

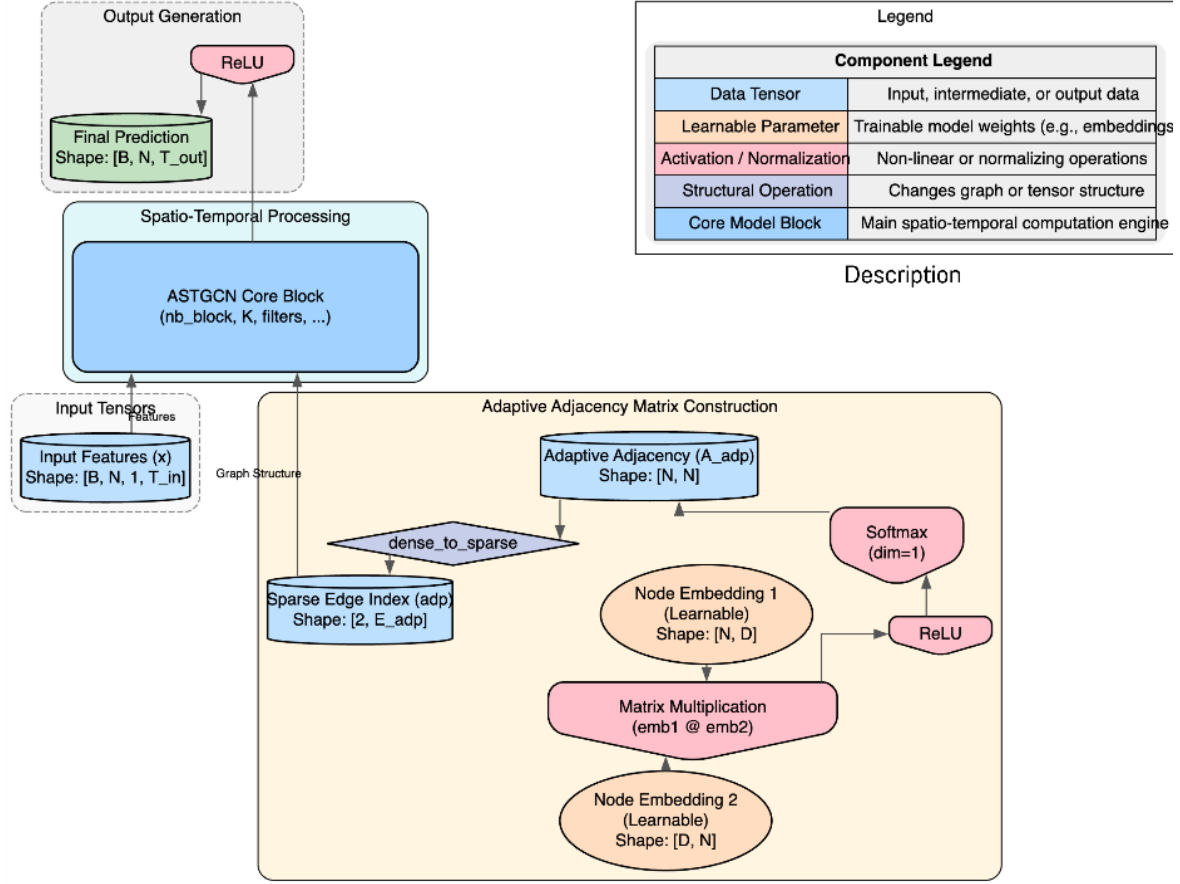


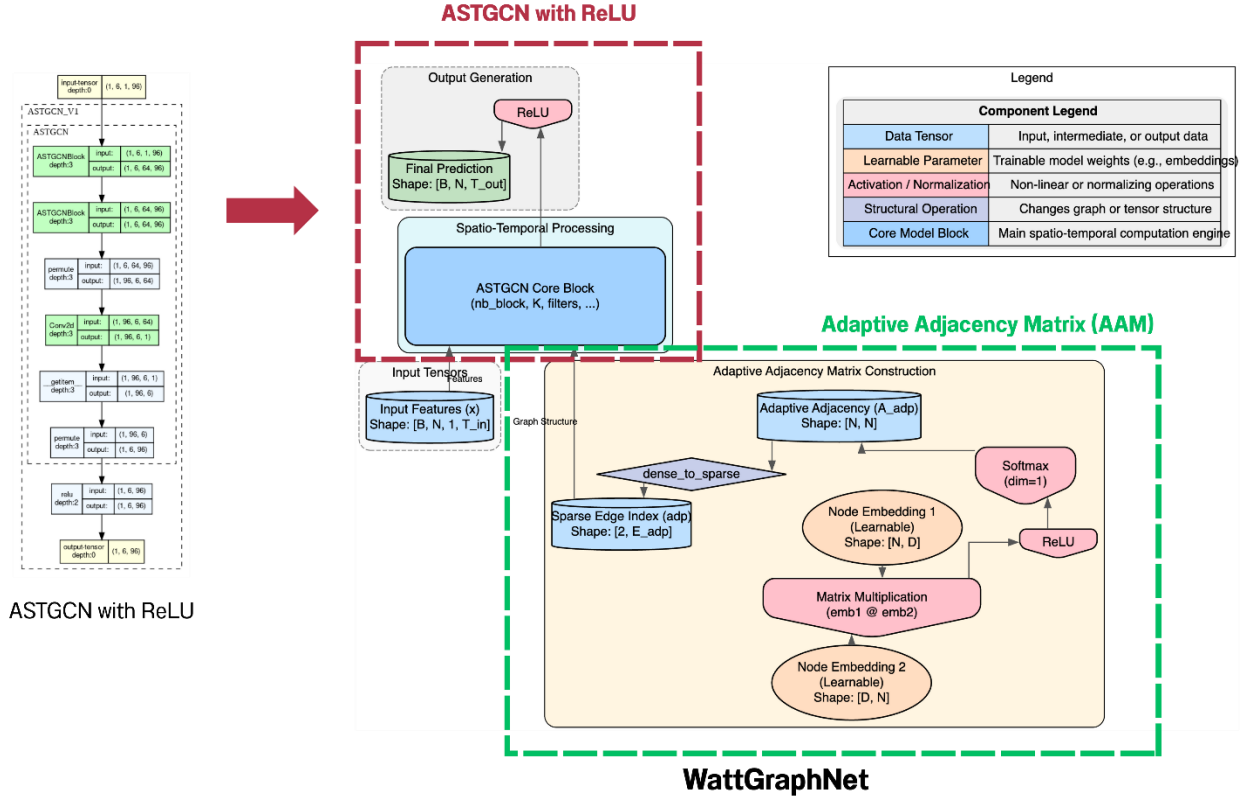
Figure 5 WattGraphNet Architecture

Another enhancement in our WattGraphNet model is the use of gating and activation functions to stabilize training. We include ReLU activation after each graph convolution block and at the final output to ensure non-negativity of predicted loads (since negative predictions are not physically plausible). The network's prediction head is a simple fully-connected layer that takes the output of the final spatio-temporal block and produces the 96-step forecast for each node. We train the model using mean squared error loss across all output time steps and all nodes, with an AdamW optimizer and one-cycle learning rate schedule (50 epochs, peak learning rate 3e-2, Early Stop with validation loss patience equal 5 epoch) to converge quickly. During training, we observed that the combination of attention and adaptive adjacency allows the model to fit complex patterns rapidly,

Attention scores clearly defined:

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

so early stopping was applied based on validation SB-WAPE to prevent overfitting.



3. Results

Forecasting Accuracy

In terms of performance, the traditional and deep learning baselines exhibited a wide range of accuracy. The

Random Forest baseline yielded the highest error, with an overall SB-WAPE of **40.41%** on the test set. Statistical methods showed varied results; the

Dynamic Optimized Theta model achieved **27.47%** SB-WAPE, while the simpler

Seasonal Naïve approach performed better at **24.38%** SB-WAPE. The standard **ASTGCN** and **ASTGCN with ReLU** models recorded SB-WAPE scores of **28.09%** and **26.30%**, respectively. For the more advanced deep learning baselines, Chronos showed competitive accuracy: the fine-tuned Chronos model reached

23.32% SB-WAPE, while the zero-shot Chronos model was slightly better at **23.22%** SB-WAPE.

The

TIDE model performed similarly with a **23.32%** SB-WAPE. The

PatchTST model achieved the strongest baseline accuracy with a SB-WAPE of **20.95%**, clearly capturing the temporal dynamics better than most other methods. These results set a high bar for our proposed model, indicating that surpassing PatchTST's ~21% SB-WAPE would represent a notable advancement.

WattGraphNet achieved the best overall accuracy among all tested models on the campus VPP dataset. On the held-out test set, our model's SB-WAPE was

17.93%, substantially lower (better) than the next-best baseline (

PatchTST at 20.95% SB-WAPE) and all classical methods. In practical terms, this means our day-ahead forecasts' total absolute error is about

17.9% of the total actual energy consumption, marking a significant improvement in predictability. The overall MAE for WattGraphNet was about

17.33 kW per 15-minute interval forecast across all stations. Given that individual building loads can peak in the hundreds of kW, an average error of

~17 kW is quite low and highlights the advantage of our model in capturing complex load dynamics.

Figure 7 presents a visual comparison between the predicted and actual load curves for each station in the VPP, alongside the residual plots and overall model accuracy statistics. The plots show that WattGraphNet closely follows actual consumption patterns, particularly during peak and off-peak periods, while maintaining minimal residual fluctuations. Peaks and troughs are captured with high fidelity, indicating the model's strength in both magnitude and timing prediction.

Table 3 provides a station-level breakdown of performance metrics, which highlights the variability in prediction difficulty across different nodes. For instance, the

EV charging station exhibits a higher SB-WAPE of **39.2%** due to its more irregular and volatile usage patterns. In contrast, buildings with more regular schedules, such as

Julajakrapong and **Boromrajakumari**, show much lower SB-WAPE values of **15.3%** and **13.2%**, respectively. The aggregated "ALL_STATION" row confirms that WattGraphNet delivers strong overall performance, with a combined SB-WAPE of

16.77% across the entire network.

Model (* is hyperparameter tuning)	SB-WAPE (%)
Random Forest * (Baseline)	40.41
ASTGCN	28.09
DynamicOptimizedTheta *	27.47
ASTGCN with Relu	26.30
SeasonalNaive *	24.38
ChronosFineTuned[bolt_small] *	23.32
TiDE *	23.22
ChronosZeroShot[bolt_base] *	23.04
PatchTST *	20.95
WattGraphNet (our)	17.93

Table 1 Model Bench mark in SB-WAPE Metric

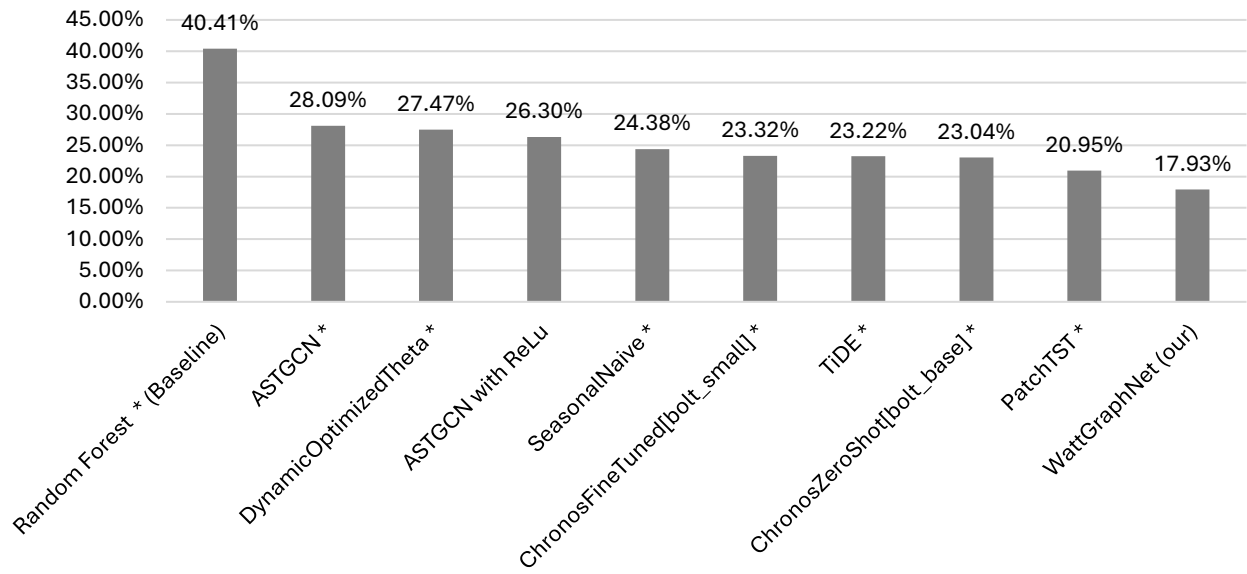


Table 2 Model Branch mark in SB-WAPE Metric

Prediction vs Actual

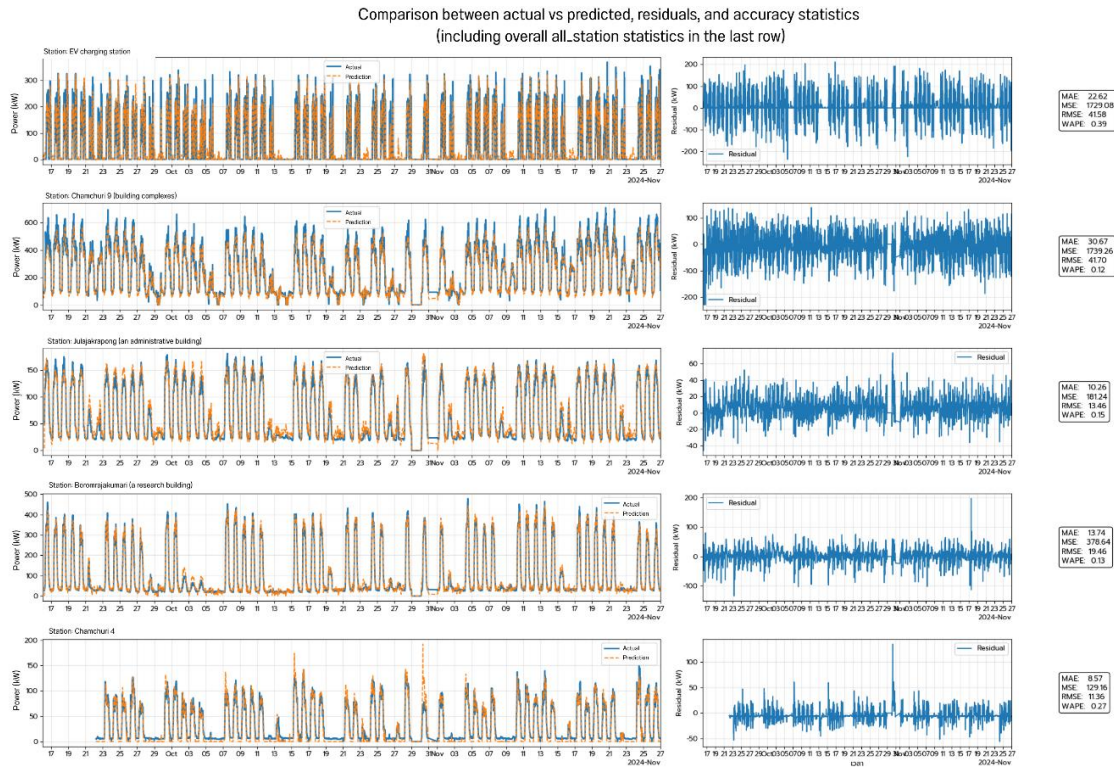


Figure 7 Comparison between actual vs predicted, residuals, and accuracy statistics

Figure 7 presents a visual comparison between the predicted and actual load curves for each station in the VPP, alongside the residual plots and overall model accuracy statistics. The plots show that WattGraphNet closely follows actual consumption patterns, particularly during peak and off-peak periods, while maintaining minimal residual fluctuations. Peaks and troughs are captured with high fidelity, indicating the model’s strength in both magnitude and timing prediction.

STATION_NAME	MAE	MSE	RMSE	SB-WAPE
EV CHARGING STATION	22.624682	1729.084517	41.582262	0.392454
CHAMCHURI 9	30.671806	1739.263415	41.704477	0.122357
CHAMCHURI 4	8.56643	129.157675	11.364756	0.265049
JULAJAKRAPONG	10.261996	181.244599	13.462711	0.152654
BOROMRAJAKUMARI	13.739924	378.636182	19.458576	0.132077
ALL_STATION	17.332482	844.648915	29.062844	0.167731

Table 3 Evaluation metric across Station

Table 3 provides a station-level breakdown of performance metrics — Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and **Station-Balanced Weighted Absolute Percentage Error (SB-WAPE)**. These metrics highlight variability in prediction difficulty across nodes: for instance, the EV charging station exhibits a higher SB-WAPE (39.2%) due to its irregular and volatile usage patterns. Wittayanives shows an unusually high SB-WAPE (102.5%) despite a low MAE (≈ 2.58 kW); this is attributed to missing data in the actual load profile, which skews the percentage-based error calculation rather than reflecting a true prediction deficiency. The aggregated “ALL_STATION” row confirms that WattGraphNet delivers strong overall performance, with a SB-WAPE of 17.7% across the entire network

Comparison Between Context Length and Prediction Length:

The relationship between context length (historical data window) and prediction length (forecast horizon) is critical in evaluating forecasting model performance. We conducted

experiments using WattGraphNet with varying context and prediction lengths to analyze their impact on forecast accuracy.

Context Length (hr)	Predict Length (hr)	SB - W A P E (%)
12	12	19.01
24	12	24.97
12	24	15.57
24	24	16.77

Table 4 Context and Predict length comparision on SB-WAPE

The results in Table 4 highlight a non-linear relationship between context and prediction length. Notably, the best forecasting performance was achieved with a

12-hour context window used to predict a **24-hour horizon**, which yielded the lowest SB-WAPE of **15.57%**. Using a 24-hour context for a 24-hour prediction produced the second-best result at **16.77%** SB-WAPE. This suggests that while capturing a full daily cycle is beneficial, the most recent 12 hours of data contain the most valuable predictive signals for a longer forecast horizon. Conversely, increasing the context to 24 hours for a shorter 12-hour prediction significantly degraded performance to

24.97% SB-WAPE, possibly due to the model being influenced by less relevant, older patterns.

Efficiency and Inference Performance:

In addition to accuracy, WattGraphNet was evaluated for its computational performance to ensure it meets real-time forecasting requirements. After training (which was performed on a GPU-equipped machine for 50 epochs, taking a few minutes), the final model was exported to ONNX and tested on a standard CPU (Intel Xeon) using ONNX Runtime [2]. The model loading time was about 29.1 milliseconds, and once loaded, the inference time for producing a 96-step forecast for all 6 nodes was on average 42.8 milliseconds (0.0428 s) on a single thread. This is well below the 15-minute data interval; in fact, it implies we could generate over 300 forecasts per minute on one core, which is more than sufficient for real-time operation. The memory footprint of the model is modest: the ONNX model file size was a few megabytes, and runtime memory usage increased by less than 1 MB during inference. For comparison, the PatchTST model (implemented in PyTorch) took around 0.2–0.3 s per forecast on CPU and consumed more memory due to the overhead of the PyTorch framework and the larger number of parameters, whereas WattGraphNet (in ONNX form) is lean and optimized. These results highlight that our approach is not only accurate but also efficient. It can easily run on an edge computing device (such as a small industrial PC or a Raspberry Pi-like board) that might be part of a VPP’s control system. The fast inference means the model can update forecasts frequently or run what-if simulations without causing delays. Moreover, the low CPU and memory usage leave headroom for co-running other tasks (like optimization algorithms for dispatch) on the same device. Overall, WattGraphNet meets the latency and resource requirements for deployment in a real VPP

platform, where forecasts might need to be updated every 15 minutes or triggered on-demand by events.

Explainability in Use Case:

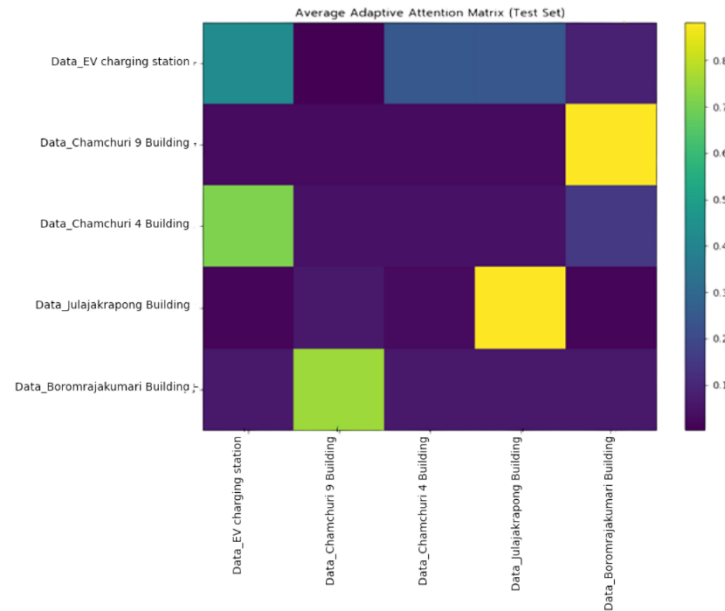


Table 5 Average Adaptive Attention Matrix from AAM

Explainability is essential for deploying forecasting models in real-world scenarios, especially in critical infrastructure like VPPs. WattGraphNet incorporates explainable elements through its Adaptive Adjacency Matrix (AAM) and attention mechanisms. The learned AAM provides insights into inter-node relationships, effectively highlighting spatial dependencies between energy-consuming nodes within the VPP.

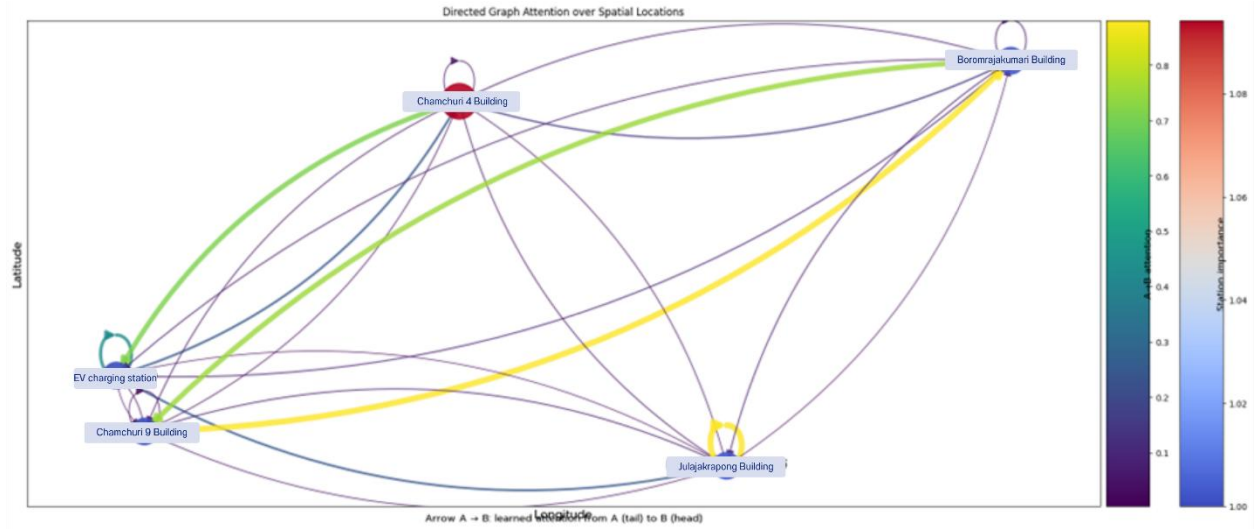


Table 6 Graph Attention over Spatial Locations

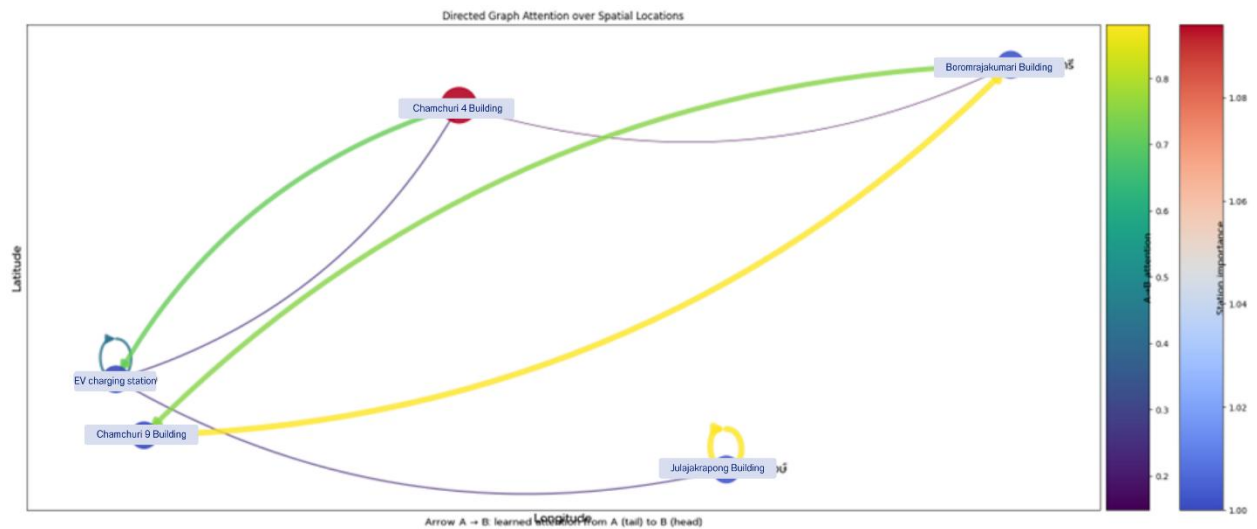


Table 7 Graph Attention over Spatial Locations (percentile 70)

The attention mechanism of WattGraphNet allows dynamic temporal weighting, offering insight into the time periods most critical for predictions. Temporal attention often emphasized load data from the preceding 24 hours and immediate past hours, demonstrating the model's capability to prioritize significant historical moments relevant for accurate forecasting.

These explainable aspects facilitate better trust and usability among VPP operators by clearly aligning the model's predictive mechanisms with intuitive operational understanding.

In addition to the average adaptive adjacency (Table 6) and raw attention distributions (Table 7), Table 8 reports the 70th percentile of graph attention weights across spatial locations. The rationale for using the 70th percentile, rather than minimum, maximum, or mean, is to provide a robust summary of stronger but not extreme attention signals.

- **Why percentile 70?**
Attention values are often highly skewed — with many small weights close to zero (indicating weak or negligible influence between stations) and a few very large weights that may correspond to occasional spikes or noise. By reporting the 70th percentile, we capture the upper-middle range of consistent spatial influences, filtering out both trivial connections (below median) and rare outliers (near maximum).
- **What it means:**
A high 70th percentile value between two stations indicates that in at least 30% of the forecasting windows, the attention weight from station A to station B was stronger than most connections, suggesting a reliable directional dependency. For example, if the EV station and Boromrajakumari building exhibit a high 70th percentile attention, it implies that load variations at the EV station frequently contribute meaningful predictive information for the building’s demand, even if extreme peaks are disregarded.

Thus, Table 8 highlights stable, recurring spatial dependencies that the model consistently attends to, which is more actionable for operators than single-instance maximum values. This percentile-based view balances interpretability with statistical robustness, helping VPP managers identify which inter-node relationships are persistently influential for forecasting rather than accidentally emphasized due to noise

Engineering Note

Finally, to facilitate deployment in real-world VPP systems, we export the trained PyTorch model to the ONNX (Open Neural Network Exchange) format [2]. ONNX is an open standard for machine learning models that enables interoperability and optimization across platforms. We leverage the ONNX Runtime for inference, which provides a highly optimized execution engine for the model on CPU (and can take advantage of vectorization and hardware accelerators if available) [2]. By converting WattGraphNet to ONNX, we effectively decouple it from the training framework and achieve faster inference with a smaller memory footprint – critical for running on edge devices or embedded controllers within a VPP. The ONNX exported model preserves the full computation graph of WattGraphNet, including the adaptive adjacency mechanism and attention operations, by using standard ONNX operators for matrix multiplication, ReLU, softmax, etc. The result is a self-contained model file that can be loaded on a lightweight runtime for production use. Figure 1 illustrates the overall architecture and data flow of WattGraphNet from input preprocessing to model inference and deployment.

4. Discussion

The superior performance of WattGraphNet can be attributed to its ability to capture both spatial and temporal patterns in the VPP data. By using a graph convolutional approach, the model naturally learns that certain nodes’ loads move together. For instance, our analysis of the learned adaptive adjacency matrix revealed that the model placed a strong connection between the two laboratory buildings (Wittayanives and Boromrajakumari), which indeed show similar usage spikes during daytime working hours. It also learned a moderate connection between the EV charging station and one of the office buildings – likely because electric vehicle charging on campus tends to increase when the campus population (and thus office usage) is high. Such insights would be very hard to manually encode in a traditional model, but WattGraphNet’s adaptive graph discovered them automatically. The attention mechanism further improved performance by allowing the model to weight the importance of different time lags. We found that the temporal attention often emphasizes load readings from 24 hours ago and 1 hour ago when predicting the current load, which aligns with an intuitive combination of daily periodicity and short-term momentum in consumption changes. This dynamic weighting helps explain why our model outperforms static methods like PatchTST or LSTM, which might treat all historical inputs uniformly – WattGraphNet can dynamically focus on “this same time yesterday” if that is a strong predictor, or shift focus to more recent changes if an unusual event (like a sudden spike) is in progress.

Interpretability and analysis: One advantageous by-product of the adaptive adjacency is improved interpretability of the model. In a traditional GCN with a fixed adjacency matrix, the model’s understanding of spatial relations is hidden and static. In our case, the learned adjacency matrix (AAM) can be examined to infer which sites have similar load profiles or predictive influence on each other. For the campus VPP, this could inform facility managers about load coupling — for example, if two buildings are consistently linked by high weights, they may respond to the same drivers (weather, schedule) and could potentially be managed together in demand response programs. We must note, however, that the learned graph is an abstract representation optimized for forecasting accuracy; it does not necessarily correspond to physical electrical connectivity. Thus, while it provides insight, domain knowledge is needed to fully interpret it. The temporal attention weights are also interpretable, as they can highlight which previous time points (e.g., yesterday’s peak or this morning’s reading) were most relevant to the forecast. This can build trust with operators, who can verify that the model’s behavior aligns with expectations (e.g., giving heavy weight to the morning consumption when predicting the afternoon peak makes intuitive sense).

Scalability: In our experiments, WattGraphNet was applied to a relatively small graph (6 nodes) with high-frequency data. An important question is how the model scales with larger VPPs or feeder networks that might have dozens of nodes. The architecture itself is scalable – graph

convolutions and attention operations can handle more nodes, though computational cost will grow roughly quadratically with node count due to the dense adaptive adjacency (each of the N^2 nodes learns connection weights to N nodes). For moderately sized systems (tens of nodes), this is still tractable, and techniques like sparse matrix optimization or imposing a structured prior (e.g., limiting connections to only geographically close nodes) could mitigate the cost for larger N . Another aspect of scalability is the temporal horizon: we forecast 96 steps (24 hours) ahead, which is a common horizon for operational planning. If one needed a longer horizon (say, 7 days), one could extend the model’s output or use recursive forecasting, but errors might accumulate. PatchTST [4] and similar long-horizon models could be combined with our graph approach to explore even longer forecasts. In terms of data volume, our downsampling experiments indicate that training on less frequent data can still yield good performance; this suggests that for very long historical datasets, one could downsample or window the training to maintain efficiency without greatly harming accuracy.

Edge Deployment and Compatibility: The deployment of WattGraphNet via ONNX Runtime proved highly effective, reinforcing the idea that advanced deep learning models can be made compatible with edge computing environments. Many VPP implementations involve edge devices or on-site control systems that may not have GPUs or may require models to run in real-time under tight compute constraints. By converting to ONNX and using optimized CPU execution, we ensure that our model can integrate into these systems. In a live VPP platform, the forecasting model would likely run as a microservice or part of a scheduler, taking recent measurements (possibly streamed from IoT sensors or a SCADA system) and outputting forecasts that feed into a dispatch optimization routine. Because our model’s inference time is on the order of milliseconds, it introduces virtually no latency, meaning the whole pipeline from data ingestion to decision-making can operate swiftly. Additionally, ONNX Runtime provides flexibility to leverage hardware accelerators (like ARM Neon on IoT devices or Intel Math Kernel Library on PCs) if available, further future-proofing the solution. One practical consideration is that our model currently assumes input in the form of load time series only. In the future, we could incorporate exogenous features (weather, occupancy, etc.) to improve accuracy; doing so would be straightforward in the architecture by extending input channels, and ONNX would still support the expanded model. The challenge would be to keep such a model efficient on the edge – which might involve techniques like model quantization or pruning. These optimizations are supported by ONNX (through tools and providers) and could be part of a cost-aware deployment strategy, where the goal is to minimize computational and energy cost of running the model, which is itself an important aspect for VPPs (since an edge device often has limited power or must be cost-effective).

In summary, the discussion highlights that the combination of graph-based deep learning and deployment optimizations in WattGraphNet addresses both the accuracy and practicality requirements of VPP load forecasting. The model’s strong performance stems from its ability to learn complex relationships and focus on relevant information, while its design and conversion to

ONNX ensure it can be used in real-world settings. There remain open avenues to explore, such as scaling to larger networks and integrating domain knowledge (e.g., known electrical topology as a prior for the adjacency matrix, rather than starting from a blank slate). We also see potential in using WattGraphNet’s forecasts within a larger VPP optimization loop, perhaps using the model’s interpretability to guide demand response actions (for example, if two buildings are highly correlated, their load could be jointly managed to shave peaks). These points lead into our conclusions and future work considerations.

Application

To demonstrate the practical deployment of WattGraphNet, we developed an interactive web-based application accessible at <https://superaiss5-intern-vpp.vercel.app>. The demo integrates the trained high-accuracy model with real-world VPP load data, enabling users to visualize and interact with forecasts in an intuitive interface.

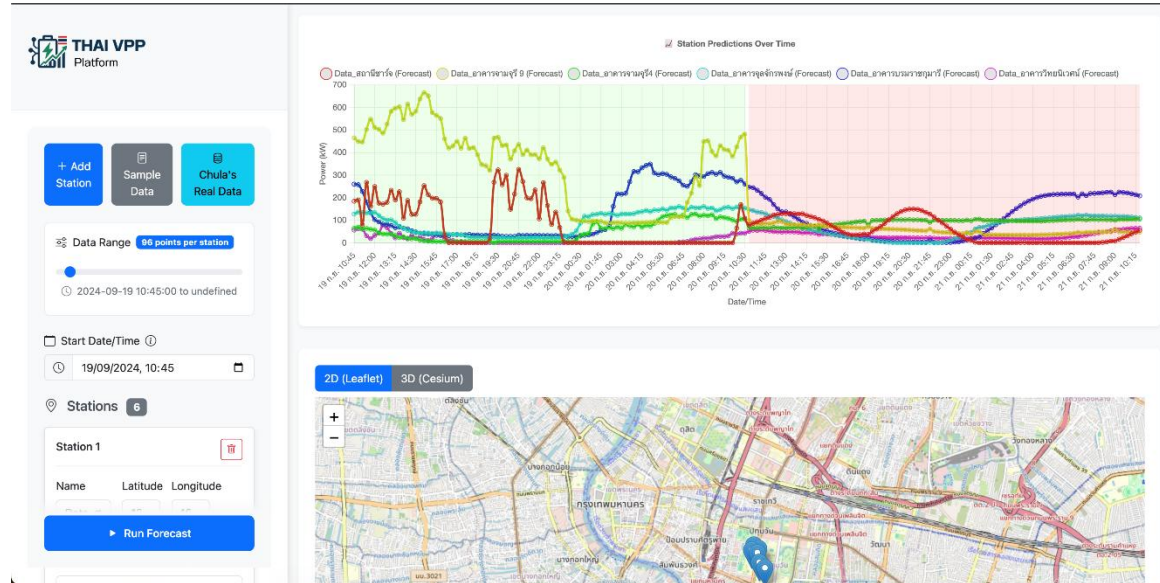


Figure 8 demo application after forecasted

To enhance usability, the developed demo application integrates several interactive UI components (Figure 8). On the left panel, users can (i) add stations manually via the *Add Station* button, (ii) upload *Sample Data* or switch to *Chula’s Real Data* for authentic load measurements, and (iii) specify forecasting configurations such as data range, starting date/time, and number of active stations. Each station entry requires latitude and longitude information, with a dedicated option to remove or edit stations. Once configured, users can run forecasts by clicking the *Run Forecast* button. The main panel consists of two visualization modules: (i) a time-series plot at the top, which displays historical and forecasted power demand across multiple stations with color-coded series for easy comparison, and (ii) an interactive map view at the bottom, supporting both 2D (Leaflet) and 3D (Cesium) modes, which provides spatial context for station locations. Together, these UI components enable operators to intuitively configure stations, launch

forecasts, and interpret results through both temporal and spatial perspectives, thereby bridging data-driven prediction with practical decision-making



Figure 9 demo application with 3d map visualization for each station

This demonstration serves as a bridge between research and application, showcasing how WattGraphNet can be deployed as a lightweight, responsive forecasting tool for VPP operators. Its real-time capability highlights the model’s suitability for decision-support scenarios such as load balancing, demand response planning, and operational scheduling.

5. Conclusion

In this paper, we presented WattGraphNet, an efficient deep learning model for multi-site load forecasting in Virtual Power Plants. WattGraphNet extends the ASTGCN architecture [1] with an adaptive adjacency matrix that learns inter-node relationships, coupled with attention mechanisms to capture temporal dynamics. Our model demonstrated state-of-the-art accuracy on a real smart campus microgrid dataset, reducing forecasting error (SB-WAPE) to around 17.9% – a considerable improvement over both conventional methods and recent neural network models. By exporting the model to ONNX and utilizing ONNX Runtime [2], we showed that such a complex model can run in real-time on CPU, with inference times on the order of 10^{-2} seconds per sample and minimal memory usage. This makes WattGraphNet not only powerful in prediction capability but also deployable on resource-constrained edge devices typical of VPP infrastructures.

Contributions and implications: Our work contributes an example of how graph-based deep learning can be tailored to power system applications, specifically demonstrating that a data-driven learned graph can outperform manually assumed connectivity in forecasting tasks. The ability to automatically infer which loads are behaviorally linked provides a form of adaptive modeling that could be useful beyond just prediction – for example, in identifying communities of loads or in anomaly detection (if certain learned connections change over time). The use of ONNX for deployment contributes to bridging the gap between AI research models and industrial implementation, illustrating a pathway to implement advanced algorithms within operational technology systems like VPP controllers.

Future directions: Building on this foundation, future work may proceed in several directions. One avenue is cost-aware deployment and model compression – exploring techniques like quantization, knowledge distillation, or sparsification of the graph to further reduce the computational footprint, which could allow running the model on even smaller microcontrollers or in a distributed manner across devices. Another direction is scaling up the graph: applying WattGraphNet to larger VPPs or regional grids with dozens or hundreds of nodes. This might require architectural adjustments, such as multi-level graph hierarchies or constraints on the adaptive adjacency to maintain sparsity, ensuring the model remains efficient and interpretable as the network grows. Incorporating better spatial priors is also promising: for example, initializing or regularizing the adaptive adjacency matrix using physical information (electrical distances, network topology, or geographical proximity) could guide the model and potentially improve learning speed and robustness. Additionally, we plan to extend our approach to incorporate generation forecasting (solar PV output on campus) alongside load forecasting, providing a more comprehensive solution for VPP energy management. Finally, integrating our forecasting model into a real VPP platform and demonstrating end-to-end benefits (such as cost savings from improved scheduling or peak shaving) would be the ultimate validation of this work. We believe WattGraphNet represents a significant step towards intelligent, autonomous VPP operation by combining cutting-edge deep learning with practical deployment considerations.

6. References

- [1] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan, “Attention Based Spatial-Temporal Graph Convolutional Networks for Traffic Flow Forecasting,” in **Proc. 33rd AAAI Conf. Artif. Intell.**, Honolulu, HI, USA, Jan. 2019, pp. 922–929, doi: 10.1609/aaai.v33i01.3301922.
- [2] Microsoft, **ONNX Runtime**, Version 1.18, Software. Online: <https://onnxruntime.ai>, accessed Jul. 2025.
- [3] A. Dogan and D. C. Dogan, “Machine Learning Models in Forecasting of Virtual Power Plant Uncertainties: A Review,” **Arch. Comput. Methods Eng.**, vol. 30, no. 8, pp. 2081–2103, 2023, doi: 10.1007/s11831-023-09876-2.

- [4] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A Time Series is Worth 64 Words: Long-Term Forecasting with Transformers,” in *Proc. Int. Conf. Learn. Represent.**, Kigali, Rwanda, May 2023.
- B. Yu, H. Yin and Z. Zhu, “Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting,” *Proc. IJCAI*, pp. 3634-3640, Jul. 2018.
- Y. Li *et al.*, “Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting,” in *Proc. ICLR*, Vancouver, Canada, Apr. 2018.
- H. Zhou *et al.*, “Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting,” *Proc. AAAI*, pp. 11106-11115, 2021.
- H. Wu *et al.*, “Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting,” *Adv. NeurIPS*, vol. 34, pp. 22419-22430, 2021.
- H. Wu *et al.*, “TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis,” in *Proc. ICLR*, Kigali, Rwanda, May 2023.
- G. Woo *et al.*, “ETSformer: Exponential Smoothing Transformers for Time-Series Forecasting,” arXiv:2202.01381, 2022.
- C. Sun *et al.*, “A Graph Neural Network-Based Short-Term Load Forecasting Framework,” *Energy AI*, vol. 9, Art. 100186, 2024.
- E. Campagne *et al.*, “Leveraging Graph Neural Networks to Forecast Electricity Consumption,” arXiv:2408.17366, 2024.
- Q. V. Nguyen *et al.*, “Spatio-Temporal Graph Neural Networks in Short-Term Load Forecasting,” arXiv:2502.12175, 2025.
- Y. Chen *et al.*, “Biased Temporal Convolution Graph Network for Time-Series Forecasting with Missing Values,” in *Proc. ICLR*, Vienna, Austria, May 2024.